

Assignment 03 - Message Passing

Problem Analysis

The assignment focuses on message passing through asynchronous actors in Java/Pekko and synchronous channel-based communication in Go. The common design constraint is avoiding shared mutable state between concurrent entities.

Exercise 1 - Smart Home Alarm System

The alarm supports five states: DISARMED, EXIT_DELAY, ARMED, ENTRY_DELAY, and ALARM. Exit and entry delays are configurable. The optional zone-based extension is implemented.

Actor Architecture

```
AlarmSystemActor
|-- ControlPanelActor
|-- KeypadActor
|-- SensorActor(front-door)
|-- SensorActor(living-motion)
|-- ...
```

AlarmSystemActor is the guardian/supervisor. ControlPanelActor is the state machine. Each state is represented by a different Pekko Behavior. KeypadActor and SensorActor model the peripherals.

State Transitions

```
DISARMED -- valid PIN --> EXIT_DELAY -- timeout --> ARMED
ARMED -- active sensor --> ENTRY_DELAY -- timeout --> ALARM
ENTRY_DELAY -- valid PIN --> DISARMED
ALARM -- valid PIN --> DISARMED
```

Timers are implemented with Behaviors.withTimers, not with explicit sleep threads. Actor macro-step semantics avoid races on internal state.

Demo

```
.\mvnw.cmd exec:java
```

```
initial: StateSnapshot[state=DISARMED, activeZones=[], sirenOn=false]
after exit delay: StateSnapshot[state=ARMED, activeZones=[perimeter], sirenOn=false]
inactive living sensor ignored: StateSnapshot[state=ARMED, activeZones=[perimeter], sirenOn=false]
entry delay started: StateSnapshot[state=ENTRY_DELAY, activeZones=[perimeter], sirenOn=false]
disarmed by keypad: StateSnapshot[state=DISARMED, activeZones=[], sirenOn=false]
```

Exercise 2 - Odds-and-Evens Game

The Go program implements an elimination tournament with $N = 2^m$ players. Each round runs all matches concurrently; winners advance after a round-level fan-in barrier.

Main elements are `playerAgent` goroutines, `runMatch` match goroutines, and `RunTournament` as coordinator. Players and matches communicate through channels; no shared memory is used for coordination.

```
cd go-odds-evens
go run . 8
go test ./...
```

Verification

```
.\mvnw.cmd test
```

```
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

```
.\mvnw.cmd package
```

Packaging completed with BUILD SUCCESS.

Go

```
gofmt -w main.go main_test.go
go test ./...
go run . 8
```

```
ok      pcd-ass03-odds-evens
Winner: P7 (id=7)
```